

Structure

Programming for Problem Solving (PPS)

GTU # 3110003

USING

{C}

Programming



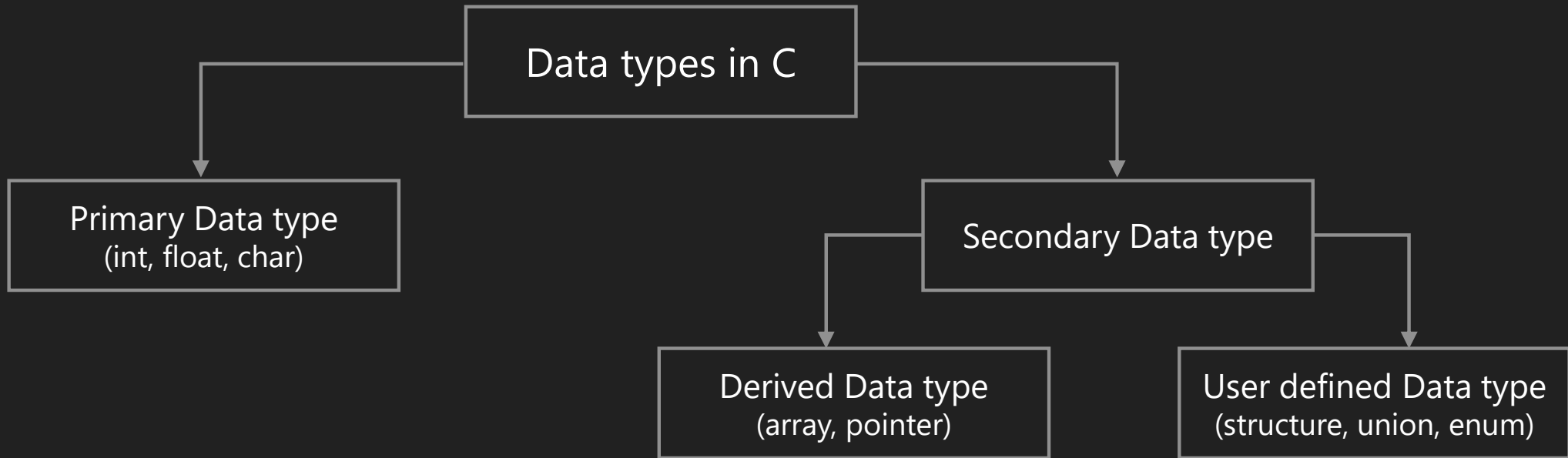
Prof. Nilesh Gambhava

Computer Engineering Department,
Darshan Institute of Engineering & Technology, Rajkot



Data Types

- ▶ Data types are defined as the data storage format that a variable can store a data.



- ▶ C language has built-in datatypes like primary and derived data types.
- ▶ But, still not all real world problems can be solved using those data types.
- ▶ We need custom datatype for different situation.

User Defined Datatype

- ▶ We need combination of various datatypes to understand different entity/object.

- ▶ **Example-1:**

→ **Book**



Title: Let Us C

Author: Yashavant Kanetkar

Page: 320

Price: 255.00

Datatype: char / string

Datatype: char / string

Datatype: int

Datatype: float

- ▶ **Example-2:**

→ **Student**



Name: ABC

Roll_No: 180540107001

CPI: 7.46

Backlog: 01

Datatype: char / string

Datatype: int

Datatype: float

Datatype: int

What is Structure?

- ▶ Structure is a collection of logically related data items of different datatypes grouped together under single name.
- ▶ Structure is a **user defined datatype**.
- ▶ Structure helps to build a complex datatype which is more meaningful than an array.
- ▶ But, an array holds similar datatype record, when structure holds different datatypes records.
- ▶ Two fundamental aspects of Structure:
 - Declaration of Structure Variable
 - Accessing of Structure Member

Syntax to Define Structure

- ▶ To define a structure, we need to use **struct** keyword.
- ▶ This keyword is reserved word in C language. We can only use it for structure and its object declaration.

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     . . .
6     memberN_declaration;
7 };
```

→ structure_name is name of custom type

→ memberN_declaration is individual member declaration

- ▶ Members can be normal variables, pointers, arrays or other structures.
- ▶ Member names within the particular structure must be distinct from one another.

Example to Define Structure

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 };
```

- ▶ You must terminate structure definition with **semicolon ;**.
- ▶ You **cannot assign value** to members inside the structure definition, it will cause **compilation error**.

Example

```
1 struct student
2 {
3     char name[30] = "ABC"; // Student Name
4     . . .
5 };
```

Create Structure variable

- ▶ A data type defines various properties about data stored in memory.
- ▶ To use any type we must declare its variable.
- ▶ Hence, let us learn how to create our custom structure type objects also known as **structure variable**.
- ▶ In C programming, there are two ways to declare a structure variable:
 1. Along with structure definition
 2. After structure definition

Create Structure Variable – Cont.

1. Declaration along with the structure definition

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     . . .
6     memberN_declaration;
7 } structure_variable;
```

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 } student1;
```


Create Structure Variable – Cont.

2. Declaration after Structure definition

Syntax

```
1 struct structure_name structure_variable;
```

Example

```
1 struct student
2 {
3     char name[30]; // Student Name
4     int roll_no; // Student Roll No
5     float CPI; // Student CPI
6     int backlog; // Student Backlog
7 };
8 struct student student1; // Declare structure variable
```

Access Structure member (data)

- ▶ Structure is a complex data type, we cannot assign any value directly to it using assignment operator.
- ▶ We must assign data to individual **structure members** separately.
- ▶ C supports two operators to access structure members, using a structure variable.
 1. Dot/period operator (.)
 2. Arrow operator (->)

Access Structure member (data) – Cont.

1. Dot/period operator (.)

- It is known as member access operator. We use **dot operator** to access members of simple structure variable.

Syntax

```
1 structure_variable.member_name;
```

Example

```
1 // Assign CPI of student1
2 student1.CPI = 7.46;
```

2. Arrow operator (->)

- In C language it is illegal to access a structure member from a pointer to structure variable using dot operator.
- We use **arrow operator** to access structure member from pointer to structure.

Syntax

```
1 pointer_to_structure->member_name;
```

Example

```
1 // Student1 is a pointer to student type
2 student1 -> CPI = 7.46;
```

Write a program to read and display student information using structure.

Program

```
1  #include <stdio.h>
2  struct student
3  {
4      char name[40]; // Student name
5      int roll; // Student enrollment
6      float CPI; // Student mobile number
7      int backlog;
8  };
9  int main()
10 {
11     struct student student1; // Simple structure variable
12     // Input data in structure members using dot operator
13     printf("Enter Student Name:");
14     scanf("%s", student1.name);
15     printf("Enter Student Roll Number:");
16     scanf("%d", &student1.roll);
17     printf("Enter Student CPI:");
18     scanf("%f", &student1.CPI);
19     printf("Enter Student Backlog:");
20     scanf("%d", &student1.backlog);
21     // Display data in structure members using dot operator
22     printf("\nStudent using simple structure variable.\n");
23     printf("Student name: %s\n", student1.name);
24     printf("Student Enrollment: %d\n", student1.roll);
25     printf("Student CPI: %f\n", student1.CPI);
26     printf("Student Backlog: %i\n", student1.backlog);
27 }
```

Output

```
Enter Student Name:aaa
Enter Student Roll Number:111
Enter Student CPI:7.89
Enter Student Backlog:0
```

```
Student using simple structure variable.
Student name: aaa
Student Enrollment: 111
Student CPI: 7.890000
Student Backlog: 0
```

Write a program to declare time structure and read two different time period and display sum of it.

Program

```
1  #include<stdio.h>
2  struct time {
3      int hours;
4      int minutes;
5      int seconds;
6  };
7  int main() {
8      struct time t1,t2;
9      int h, m, s;
10     //1st time
11     printf ("Enter 1st time.");
12     printf ("\nEnter Hours: ");
13     scanf ("%d",&t1.hours);
14     printf ("Enter Minutes: ");
15     scanf ("%d",&t1.minutes);
16     printf ("Enter Seconds: ");
17     scanf ("%d",&t1.seconds);
18     printf ("The Time is
19     %d:%d:%d",t1.hours,t1.minutes,t1.seconds);
20     //2nd time
21     printf ("\n\nEnter the 2nd time.");
22     printf ("\nEnter Hours: ");
23     scanf ("%d",&t2.hours);
24     printf ("Enter Minutes: ");
25     scanf ("%d",&t2.minutes);
26     printf ("Enter Seconds: ");
```

```
27     scanf ("%d",&t2.seconds);
28     printf ("The Time is
29     %d:%d:%d",t2.hours,t2.minutes,t2.seconds);
30     h = t1.hours + t2.hours;
31     m = t1.minutes + t2.minutes;
32     s = t1.seconds + t2.seconds;
33     printf ("\nSum of the two time's is
34     %d:%d:%d",h,m,s);
35     return 0;
36 }
37 }
```

Output

```
Enter 1st time.
Enter Hours: 1
Enter Minutes: 20
Enter Seconds: 20
The Time is 1:20:20
```

```
Enter the 2nd time.
Enter Hours: 2
Enter Minutes: 10
Enter Seconds: 10
The Time is 2:10:10
Sum of the two time's is 3:30:30
```

Array of Structure

- ▶ It can be defined as the collection of multiple structure variables where each variable contains information about different entities.
- ▶ The array of structures in C are used to store information about **multiple entities of different data types**.

Syntax

```
1 struct structure_name
2 {
3     member1_declaration;
4     member2_declaration;
5     ...
6     memberN_declaration;
7 } structure_variable[size];
```

Write a program to read and display N student information using array of structure.

Program

```
1  #include<stdio.h>
2  struct student {
3      char name[20];
4      int rollno;
5      float cpi;
6  };
7  int main( ) {
8      int i,n;
9      printf("Enter how many records u want to store : ");
10     scanf("%d",&n);
11     struct student sarr[n];
12     for(i=0; i<n; i++)
13     {
14         printf("\nEnter %d record : \n",i+1);
15         printf("Enter Name : ");
16         scanf("%s",sarr[i].name);
17         printf("Enter RollNo. : ");
18         scanf("%d",&sarr[i].rollno);
19         printf("Enter CPI : ");
20         scanf("%f",&sarr[i].cpi);
21     }
22     printf("\n\tName\tRollNo\tMarks\t\n");
23     for(i=0; i<n; i++) {
24         printf("\t%s\t\t%d\t\t%.2f\t\n", sarr[i].name,
25             sarr[i].rollno, sarr[i].cpi);
26     }
27     return 0;
28 }
```

Output

Enter how many records u want to store : 3

Enter 1 record :

Enter Name : aaa

Enter RollNo. : 111

Enter CPI : 7.89

Enter 2 record :

Enter Name : bbb

Enter RollNo. : 222

Enter CPI : 7.85

Enter 3 record :

Enter Name : ccc

Enter RollNo. : 333

Enter CPI : 8.56

Name	RollNo	Marks
aaa	111	7.89
bbb	222	7.85
ccc	333	8.56

Write a program to declare time structure and read two different time period and display sum of it using function.

Program

```
1  #include<stdio.h>
2  struct Time {
3      int hours;
4      int minutes;
5      int seconds;
6  };
7  struct Time input(); // function declaration
8  int main()
9  {
10     struct Time t;
11     t=input();
12     printf("Hours : Minutes : Seconds\n %d : %d : %d",t.hours,t.minutes,t.seconds);
13     return 0;
14 }
15 struct Time input() // function definition
16 {
17     struct Time tt;
18     printf ("Enter Hours: ");
19     scanf ("%d",&tt.hours);
20     printf ("Enter Minutes: ");
21     scanf ("%d",&tt.minutes);
22     printf ("Enter Seconds: ");
23     scanf ("%d",&tt.seconds);
24     return tt; // return structure variable
25 }
26 }
```

Output

```
Enter Hours: 1
Enter Minutes: 20
Enter Seconds: 20
Hours : Minutes : Seconds
1 : 20 : 20
```


Structure using Pointer

- ▶ Reference/address of structure object is passed as function argument to the definition of function.

Program

```
1  #include <stdio.h>
2  struct student {
3      char name[20];
4      int rollno;
5      float cpi;
6  };
7  int main()
8  {
9      struct student *studPtr, stud1;
10     studPtr = &stud1;
11     printf("Enter Name: ");
12     scanf("%s", studPtr->name);
13     printf("Enter RollNo: ");
14     scanf("%d", &studPtr->rollno);
15     printf("Enter CPI: ");
16     scanf("%f", &studPtr->cpi);
17     printf("\nStudent Details:\n");
18     printf("Name: %s\n", studPtr->name);
19     printf("RollNo: %d", studPtr->rollno);
20     printf("\nCPI: %f", studPtr->cpi);
21     return 0;
22 }
```

Output

```
Enter Name: ABC
Enter RollNo: 121
Enter CPI: 7.46

Student Details:
Name: ABC
RollNo: 121
CPI: 7.460000
```

Nested Structure

- ▶ When a **structure contains another structure**, it is called **nested structure**.
- ▶ For example, we have two structures named **Address** and **Student**. To make Address nested to Student, we have to define Address structure before and outside Student structure and create an object of Address structure inside Student structure.

Syntax

```
1 struct structure_name1
2 {
3     member1_declaration;
4     member2_declaration;
5     ...
6     memberN_declaration;
7 };
8 struct structure_name2
9 {
10     member1_declaration;
11     member2_declaration;
12     ...
13     struct structure1 obj;
14 };
```

Write a program to read and display student information using nested of structure.

Program

```
1  #include<stdio.h>
2  struct Address
3  {
4      char HouseNo[25];
5      char City[25];
6      char PinCode[25];
7  };
8  struct Student
9  {
10     char name[25];
11     int roll;
12     float cpi;
13     struct Address Add;
14 };
15 int main()
16 {
17     int i;
18     struct Student s;
19     printf("\n\tEnter Student Name : ");
20     scanf("%s",s.name);
21     printf("\n\tEnter Student Roll Number : ");
22     scanf("%d",&s.roll);
23     printf("\n\tEnter Student CPI : ");
24     scanf("%f",&s.cpi);
25     printf("\n\tEnter Student House No : ");
26     scanf("%s",s.Add.HouseNo);
```

```
27     printf("\n\tEnter Student City : ");
28     scanf("%s",s.Add.City);
29     printf("\n\tEnter Student Pincode : ");
30     scanf("%s",s.Add.PinCode);
31     printf("\nDetails of Students");
32     printf("\n\tStudent Name : %s",s.name);
33     printf("\n\tStudent Roll Number :
34 %d",s.roll);
35     printf("\n\tStudent CPI : %f",s.cpi);
36     printf("\n\tStudent House No :
37 %s",s.Add.HouseNo);
38     printf("\n\tStudent City :
39 %s",s.Add.City);
40     printf("\n\tStudent Pincode :
41 %s",s.Add.PinCode);
42     return 0;
43 }
```

Output

```
Details of Students
Student Name : aaa
Student Roll Number : 111
Student CPI : 7.890000
Student House No : 39
Student City : rajkot
Student Pincode : 360001
```

Practice Programs

1. Define a structure data type called `time_struct` containing three member's integer hours, minutes, second. Develop a program that would assign values to individual member and display the time in following format : HH:MM:SS
2. WAP to create structure of book with book title, author name, publication, and price. Read data of n books and display them.
3. Define a structure `Person` that would contain person name, date of joining, and salary using this structure to read this information of 5 people and print the same on screen.
4. Define a structure `time_struct` containing three member's integer hour, integer minute and integer second. WAP that would assign values to the individual number and display the time in the following format: 16: 40: 51.
5. Define a structure `cricket` that will describe the following information:
 - Player name
 - Team name
 - Batting average
6. Using `cricket`, declare an array `player` with 50 elements and WAP to read the information about all the 50 players and print team wise list containing names of players with their batting average.
7. Define a structure `student_record` to contain name, branch, and total marks obtained. WAP to read data for 10 students in a class and print them.



Thank you

